RepositoryManager — Library Documentation

A small class library for storing and retrieving string content (JSON or XML) identified by a unique name. Everything lives in the `RepositoryManager` namespace.

Overview

The library is built around four main pieces:

Class / Interface	Role
`RepositoryManager`	The public-facing entry point. Handles all register / retrieve / deregister operations.
`IStorageBackend<TContent>`	Interface that abstracts where data actually lives. Swap it out without touching anything else.
`InMemoryStorageBackend<TContent>`	Default implementation — keeps everything in a `ConcurrentDictionary`.
`IContentValidator`	Interface for validating content before it gets stored.
`JsonContentValidator` / `XmlContentValidator`	Concrete validators for JSON and XML.
`RepositoryItem<TContent>`	Simple wrapper that pairs a content value with its type.
`ItemType`	Enum that maps `Json = 1` and `Xml = 2`.

Getting Started

The simplest way to use the library — no configuration needed:

```
var repo = new RepositoryManager();

repo.Register("config", """{"theme":"dark"}""", (int)ItemType.Json);

string content = repo.Retrieve("config");   // {"theme":"dark"}
int type       = repo.GetType("config");    // 1

repo.Deregister("config");
```

If you need a custom storage backend (e.g. a database or a mock for tests):

```
var repo = new RepositoryManager(new MyCustomStorageBackend());
```

Public API

`Initialize()`

```
public void Initialize()
```

Marks the repository as initialised. Can only be called **once** — calling it a second time throws `InvalidOperationException`. Note that you don't actually *have* to call this before using the repository; the constructor already sets everything up. It's mainly there as an explicit lifecycle hook if your application needs one.

`Register(string itemName, string itemContent, int itemType)`

Validates and stores an item under the given name.

- `itemName` — unique key; cannot be null or empty.
- `itemContent` — the raw JSON or XML string.
- `itemType` — `1` for JSON, `2` for XML.

The content is validated against the declared type before being stored, so passing JSON content with `itemType = 2` (XML) will fail.

Exceptions:

Condition	Exception
`itemName` is null or empty	`ArgumentException`
`itemContent` is null	`ArgumentNullException`
`itemType` is not 1 or 2	`ArgumentException`
Content fails format validation	`ArgumentException`
A item with that name already exists	`InvalidOperationException`

`Retrieve(string itemName)`

```
public string Retrieve(string itemName)
```

Returns the raw content string that was stored under `itemName`. Throws `KeyNotFoundException` if nothing is registered under that name.

`GetType(string itemName)`

```
public int GetType(string itemName)
```

Returns the type integer (`1` = JSON, `2` = XML) of the stored item. Throws `KeyNotFoundException` if the name doesn't exist.

`Deregister(string itemName)`

```
public void Deregister(string itemName)
```

Removes the item from the repository. After this, the name is free to be registered again. Throws `KeyNotFoundException` if the name doesn't exist.

How the Pieces Fit Together

```
RepositoryManager
    │
    ├── IStorageBackend<string>          (injected, defaults to
InMemoryStorageBackend)
    │       └── InMemoryStorageBackend   (ConcurrentDictionary under the hood)
    │
    └── IContentValidator (one per type, keyed by ItemType int)
            ├── JsonContentValidator     (uses System.Text.Json)
            └── XmlContentValidator      (uses System.Xml)
```

When `Register` is called:

1. Arguments are validated (null checks, type range check).
2. The appropriate `IContentValidator` is looked up by `itemType`.
3. The content string is validated — if it doesn't parse, an `ArgumentException` is thrown.
4. `IStorageBackend.TryAdd` is called. If the key already exists, it returns `false` and `Register` throws `InvalidOperationException`.

When `Retrieve` or `GetType` is called:

1. `IStorageBackend.TryGet` is called.
2. If the key isn't found, `KeyNotFoundException` is thrown.
3. Otherwise the content or type is returned from the `RepositoryItem`.

Thread Safety

All public methods are safe to call from multiple threads concurrently:

- `InMemoryStorageBackend` uses `ConcurrentDictionary`, so all storage operations are atomic.

- `TryAdd` guarantees that if two threads try to register the same name at the same time, exactly one succeeds and the other gets `InvalidOperationException` — no data is silently overwritten.
- `Initialize()` uses a `lock` to ensure it runs at most once even under concurrent calls.

Extending the Library

Custom storage backend — implement `IStorageBackend<string>` and inject it via the constructor:

```
public class SqlStorageBackend : IStorageBackend<string>
{
    public bool TryAdd(string key, RepositoryItem<string> item) { /* ... */ }
    public bool TryGet(string key, out RepositoryItem<string>? item) { /* ... */ }
    public bool TryRemove(string key) { /* ... */ }
}

var repo = new RepositoryManager(new SqlStorageBackend());
```

New content type — add a value to `ItemType`, implement `IContentValidator`, and register the validator in the `RepositoryManager` constructor's dictionary. No other changes needed.

Error Reference

Exception	When it's thrown
`ArgumentException`	Null/empty name, unknown item type, or content fails format validation
`ArgumentNullException`	`itemContent` is null
`InvalidOperationException`	Duplicate name on `Register`, or `Initialize()` called more than once
`KeyNotFoundException`	Name not found in `Retrieve`, `GetType`, or `Deregister`